UrbanBites

Gestionarea unui restaurant

Nume proiect	UrbanBites
Nume document	Architectural Design Document
Autori	Buțco D-F. Dumitru-Constantin Brînză C. Denis-Ștefan Buzan C. Alexandra Maria Simiuc G. Alexandru Vasilca V. Rareș-Mihai
Reviewed by	-
Status	Finalizat

Versiuni

Dată	Versiune	Modificări
09.10.2025	0.1	Structura documentului
14.10.2025	0.2	Am ajuns pana la Concluzie
22.10.2025	1.0	Realizarea primei versiuni complete

Cuprins

1. **Introducere**
    1.1 Scopul documentului
    1.2 Viziunea proiectului
    1.3 Definiții și abrevieri

2. **Prezentarea generală a sistemului**
    2.1 Componente principale
    2.2 Actorii sistemului

3. **Arhitectura Top Level**
    3.1 Identificarea componentelor
    3.2 Serverul de servicii
    3.3 Relațiile și comunicarea dintre componente

4. **Descrierea componentelor**
    - Frontend
    - Backend
    - Baza de date
    - Serviciul de notificări
    - Endpoints

5. **Concluzie**

1. Introducere

1.1 Scopul documentului

Arhitectura sistemului **UrbanBites**, o platformă menită să gestioneze rezervările și operațiunile unui restaurant contemporan.
ADD-ul oferă o imagine generală a structurii aplicației, a componentelor principale, a relațiilor dintre acestea și a alegerilor arhitecturale care susțin dezvoltarea și întreținerea sistemului.

1.2 Viziunea proiectului

Scopul proiectului **UrbanBites** este de a îmbunătăți modul în care sunt gestionate restaurantele, oferind o platformă centralizată pentru rezervări, gestionarea meniurilor și facilitarea interacțiunilor dintre clienți și angajați.
Soluția este următoarea:

- să înlocuiască modalitățile tradiționale de rezervare, cum ar fi telefonul, documentele etc.
- să ofere clienților o interfață ușor de înțeles pe platformele web și mobile
- să permită managerilor de restaurant să gestioneze mai bine rezervările, meniurile și datele clienților.

1.3 Definiții și abrevieri

UB ~ UrbanBites
GIO ~ Grup infracțional organizat

2. Prezentarea generală a sistemului

2.1 Componentele principale ale sistemului:

1. **Frontend**

Interfața cu care interacționează utilizatorii platformei **UrbanBites** este dezvoltată folosind React și TypeScript, dar și HTML/CSS pentru structurarea și utilizarea paginilor web.

React este unul dintre cele mai populare framework-uri front-end, permițând dezvoltarea de aplicații dinamice și responsive.

TypeScript, un superset al JavaScript, îmbunătățește calitatea codului prin prevenirea erorilor comune.

Frontend-ul oferă utilizatorilor și administratorilor următoarele funcționalități:

- autentificare și înregistrare
- vizualizare meniuri și mese disponibile
- realizare și vizualizare rezervări
- notificări vizuale și mesaje de confirmare sau eroare

2. **Backend**

Serverul aplicației este implementat folosind Java și framework-ul Spring Boot, care oferă suport pentru dezvoltarea rapidă de aplicații web și servicii REST. Backend-ul gestionează logica de business a platformei, inclusiv:

- procesarea rezervărilor
- gestionarea meniurilor
- validarea datelor
- autentificare și autorizare (JWT)
- monitorizare și logare

Arhitectura backend-ului se bazează pe modelul REST (REpresentational State Transfer), ceea ce permite crearea de API-uri slab cuplate, scalabile și ușor de integrat cu alte servicii.

Serviciile sunt organizate conform modelului MVC (Model-View-Controller):

Model: definește structura datelor și interacțiunea cu baza de date

Controller: primește și răspunde la cererile HTTP

View: serializează și trimite datele către frontend

3. **Baza de date**

Platforma utilizează PostgreSQL, un sistem de gestionare a bazelor de date relaționale robust și performant. Structura bazei de date asigură integritatea datelor și permite gestionarea eficientă a relațiilor între entități precum utilizatori, restaurante, mese, meniuri și rezervări.

4. **Servicii de mesagerie și notificări**

Pentru trimiterea notificărilor asincrone, platforma folosește RabbitMQ, care permite comunicarea între componente fără a bloca execuția principală a aplicației. Prin acest serviciu, clienții primesc email-uri de confirmare pentru rezervările realizate, iar actualizările importante sunt propagate eficient către utilizatori.

5. **Deployment**

Deployment-ul platformei **UrbanBites** este realizat folosind Docker, permițând containerizarea serviciilor web și izolarea acestora pe procese separate. Fiecare serviciu expune un port specific pentru a putea fi accesat și configurat independent, facilitând scalabilitatea și mentenanța platformei.

2.2 Actorii sistemului:

1. Utilizatorii:

- utilizatorii sunt persoanele care doresc sa facă o rezervare online la restaurantele partenere. Aceștia au acces la platformă printr-o interfață prietenoasă, unde pot:

crea conturi

să vizualizeze meniurile restaurantului

să verifice mesele libere pe care le pot rezerva

să trimită mesaje restaurantelor pentru a cere informații suplimentare (dacă avem timp)

2. Admin restaurant:

- gestionează datele despre restaurant (mese, meniuri, personal)
- poate vizualiza și aproba/reprograma rezervări
- actualizează meniul și configurarea meselor
- accesează rapoarte despre activitatea restaurantului

3. Administrator global (System Admin) (daca avem timp)

- actorul care are acces la întreaga platformă **UrbanBites**, nu doar la un restaurant anume:

gestionează conturile de administratori de restaurant;
monitorizează activitatea sistemului (loguri, performanță, utilizatori activi);
poate interveni în cazul unor probleme tehnice sau fraude;
asigură mentenanța generală a platformei (backup, actualizări, securitate).

3. Arhitectura Top Level

3.1 Identificarea componentelor

Arhitectura sistemului **UrbanBites** este de tip multi-layer, fiind compusă din mai multe module independente care colaborează prin API-uri REST și servicii de mesagerie.

UI Layer (Frontend)

Reprezintă interfața cu utilizatorul, realizată în React și TypeScript. Permite vizualizarea meniurilor, a meselor disponibile și efectuarea rezervărilor. Comunicarea cu backend-ul se face prin cereri HTTP.

Application Layer (Backend Services)

Implementat în Java și Spring Boot, gestionează logica aplicației: rezervări, meniuri, utilizatori și notificări. Oferă servicii REST pentru interfață.

Database Layer

Asigură stocarea și integritatea datelor prin PostgreSQL. Gestionează entitățile principale ale sistemului: utilizatori, restaurante, mese, meniuri și rezervări.

Messaging & Notification Service

Folosește RabbitMQ pentru trimiterea notificărilor asincrone (confirmări de rezervare, alerte administrative), fără a bloca fluxul principal al aplicației.

Authentication Service

Responsabil de autentificare și autorizare, bazat pe token-uri JWT. Controlează accesul la resurse și rolurile utilizatorilor.

External APIs

Include servicii externe (e-mail, plată online) integrate prin API-uri REST, utilizate pentru extinderea funcționalităților aplicației.

3.2 Server-ul de servicii

Server-ul de servicii reprezintă nucleul aplicației UrbanBites, implementat în Java folosind framework-ul Spring Boot. Acesta urmează arhitectura MVC (Model–View–Controller) și expune API-uri REST prin care gestionează operațiile principale: rezervări, meniuri, utilizatori și notificări. Server-ul include un API Gateway care direcționează cererile către serviciile corespunzătoare și asigură securitatea accesului prin token-uri JWT.

3.3 Relațiile și comunicarea dintre componente

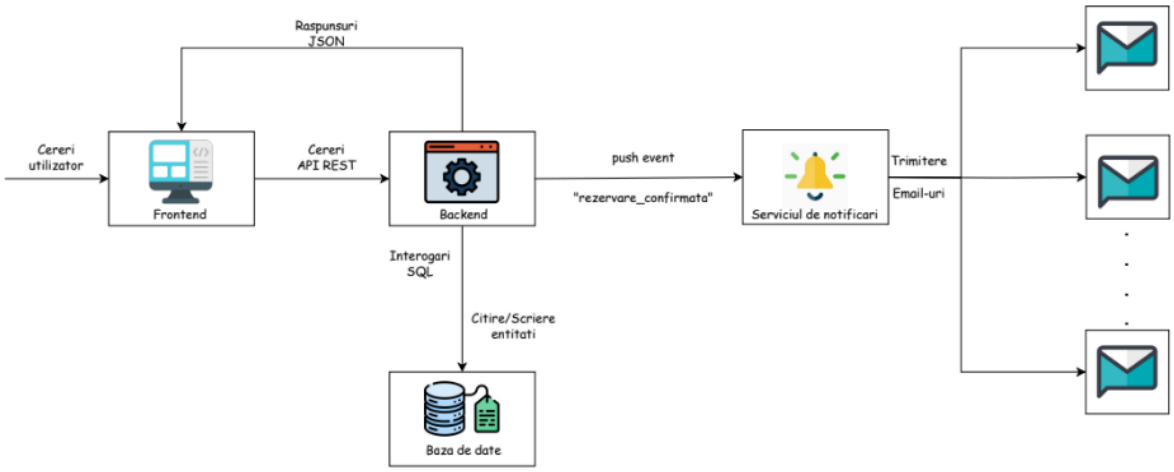

Fig. 1 Împărțirea pe servicii a server-ului de servicii web

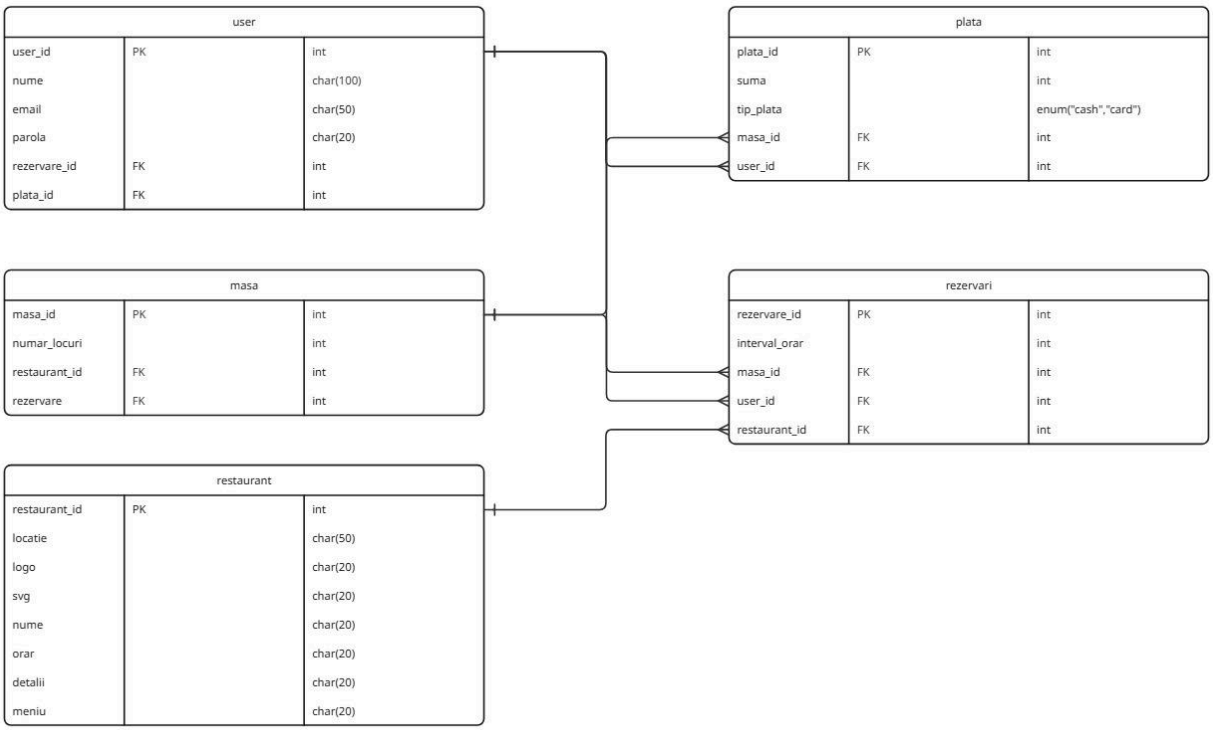

Fig. 2 Schema bazei de date

4. Descrierea componentelor

Frontend

Input: Cereri de la utilizator (login, rezervare, vizualizare meniu)

Output: Pagini dinamice, mesaje de confirmare, erori, notificari vizuale

Functionalitati:

- autentificare / inregistrare
- vizualizare meniuri și mese disponibile
- realizare rezervare
- vizualizare istoric rezervări

Backend

Input: Cereri API REST (HTTP POST/GET/PUT/DELETE)
Output: Răspunsuri JSON către frontend
Functionalitati:

- procesare rezervari
- gestionare meniuri
- validare date
- autentificare si autorizare
- logare și monitorizare

Baza de date

Entitati principale:

User(user_id, nume, email, parola, rezervare_id, plata_id)
Restaurant(restaurant_id, locatie, logo, svg, nume, orar, detalii, meniu)
Masa(masa_id, numar_locuri, restaurant_id, rezervare)
Plata(plata_id, suma, tip_plata, masa_id, user_id)
Reservation(rezervare_id, interval_orar, masa_id, user_id, restaurant_id)
<u>Relații:</u>
Un user poate avea mai multe rezervări și plăți
O masă poate avea mai multe rezervări
Un restaurant poate avea mai multe rezervări

Serviciul de notificari (daca apucam)

Trimite e-mailuri pentru confirmări de rezervare
Rulează asincron prin intermediul cozii de mesaje

Endpoints:

GET /api/reservations – Pentru preluarea tuturor rezervărilor existente.
GET /api/reservations/{id} – Pentru preluarea detaliilor unei rezervări specifice.
POST /api/reservations – Pentru crearea unei noi rezervări.
DELETE /api/reservations/{id} – Pentru ștergerea unei rezervări din sistem.
GET /api/users – Pentru preluarea tuturor utilizatorilor înregistrați.
GET /api/tables – Pentru listarea meselor disponibile.
GET /api/tables/{id} – Pentru vizualizarea detaliilor unei mese.
POST /api/notifications/send – Pentru trimiterea unei notificări de confirmare.

Și altele, odată ca începem sa dezvoltam aplicația :)

5. Concluzie

Platforma **UrbanBites** își propune să aducă un nou mod prin care restaurantele gestionează rezervările și operațiunile zilnice, oferind o soluție digitală integrată, simplă și eficientă. Prin centralizarea informațiilor legate de meniuri, mese disponibile și rezervări, UrbanBites va contribui la îmbunătățirea experienței atât pentru clienți, cât și pentru administratori.

Clienții vor beneficia de o interfață prietenoasă, accesibilă atât pe web, cât și pe mobil, care facilitează găsirea rapidă a restaurantelor, vizualizarea meniurilor actualizate și realizarea de rezervări în timp real. Pe de altă parte, managerii și personalul restaurantului vor avea la dispoziție un sistem intuitiv pentru gestionarea rezervărilor, actualizarea meniurilor și analiza activității prin rapoarte dedicate.